

Memoria descriptiva de la aplicación de escritorio SisBib

1. Introducción

La presente memoria descriptiva documenta el funcionamiento, instalación y uso de la aplicación de escritorio SisBib desarrollada bajo el framework Qt y utilizando una base de datos SQLite.

SisBib es un sistema de gestión bibliotecaria orientado a la administración de préstamos de materiales, permitiendo registrar nuevos préstamos verificando la disponibilidad de materiales y los límites correspondientes, así como gestionar su devolución. Además, la aplicación incluye módulos para la gestión de usuarios y la gestión de materiales, ofreciendo funcionalidades de creación, modificación y eliminación de registros. El objetivo principal del sistema es facilitar el control interno de una biblioteca, optimizando los procesos y asegurando la consistencia de la información almacenada.

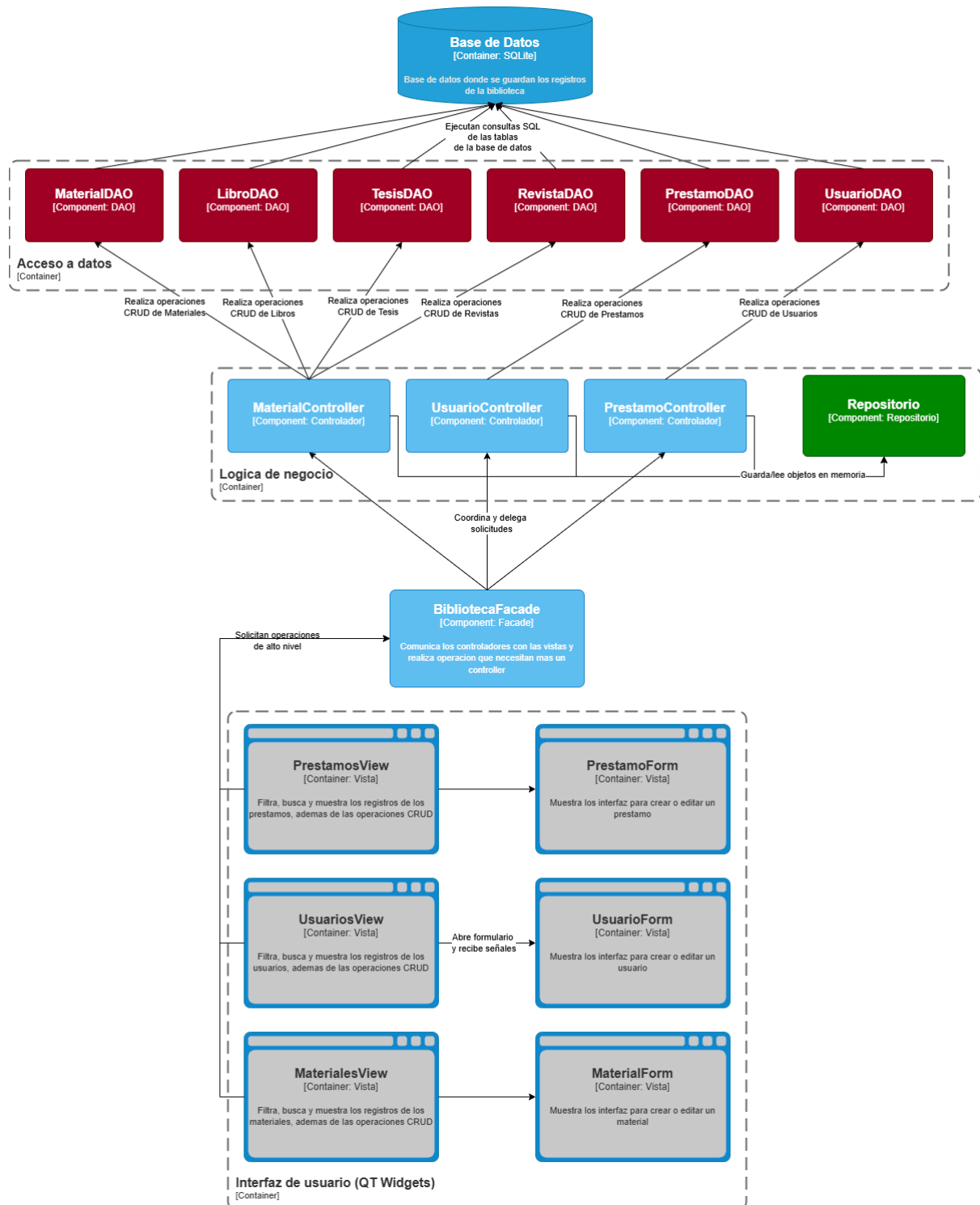
2. Arquitectura y plataforma tecnológica

2.1 Arquitectura general del sistema

SisBib es una aplicación de escritorio desarrollada en C++ utilizando el framework Qt 6, siguiendo la arquitectura Modelo–Vista–Controlador (MVC) para organizar la lógica del sistema. El sistema utiliza una base de datos SQLite, integrada localmente en la aplicación, lo que permite operar sin necesidad de un servidor externo.

La aplicación está diseñada para ejecutarse en equipos con sistema operativo Windows 10 o superior (64 bits) y requiere las bibliotecas de ejecución de Qt correspondientes a su versión (QtCore, QtGui, QtWidgets, QSql, entre otras).

El entorno de ejecución incluye el motor de base de datos SQLite 3, manejado a través del módulo Qt SQL, que permite la conexión y operaciones CRUD sobre los datos de usuarios, materiales y préstamos.



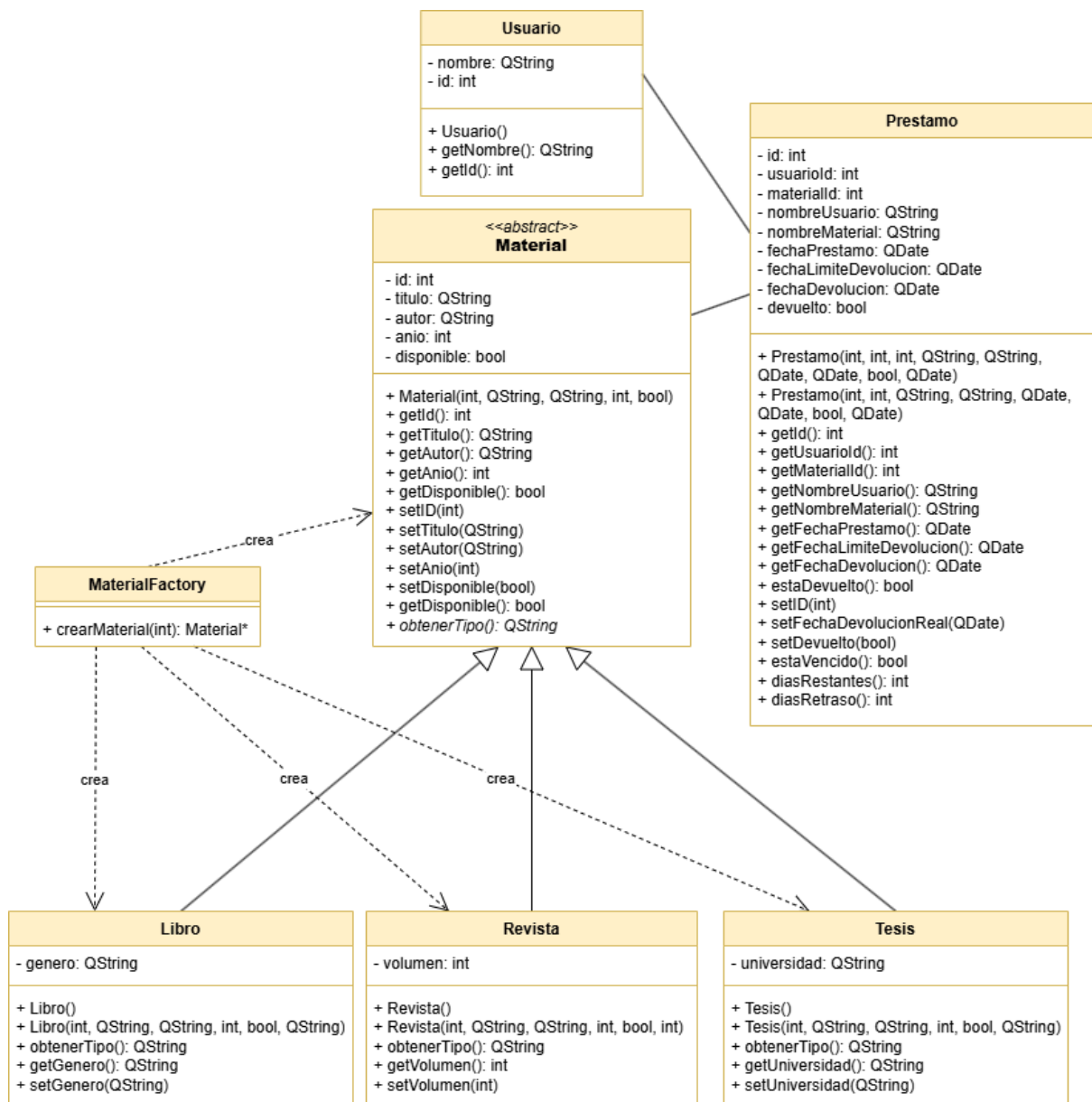
El sistema utiliza una arquitectura multicapa basada en el patrón Modelo–Vista–Controlador (MVC), reforzada con componentes DAO para acceso a datos, un repositorio en memoria para optimización y una fachada que simplifica la comunicación entre la interfaz y la lógica de negocio. La base de datos relacional almacena la información persistente de usuarios, materiales y préstamos.

2.2. Modelo del dominio

El modelo del dominio representa las entidades principales del sistema y las relaciones entre ellas, reflejando la estructura conceptual del problema que se busca resolver. Estas entidades se diseñan como clases simples que encapsulan datos y reglas mínimas relacionadas con su estructura y comportamiento.

Responsabilidades de las clases de dominio

- **Encapsular datos:** cada clase define los atributos relevantes para representar su entidad. Por ejemplo, Usuario tiene nombre e ID, Material tiene título, autor, año y disponibilidad.
- **Definir reglas básicas:** las clases pueden incluir métodos mínimos relacionados con su estado o comportamiento, como Prestamo::estaVencido() o Material::getDisponible().
- **Transportar información entre capas:** los objetos de dominio son utilizados por los Controllers, la Fachada y los DAOs, permitiendo operaciones de lectura, escritura y filtrado sin exponer la lógica interna de la base de datos.



En este sistema, las entidades del dominio son:

Usuario: Representa a las personas que pueden tomar prestados materiales de la biblioteca.

Material: Representa cualquier recurso disponible en la biblioteca, con especializaciones: Libro, Revista y Tesis.

Préstamo: Representa el registro de un material prestado a un usuario, con información sobre fechas de préstamo, y estado de devolución.

Herencia y Polimorfismo

La clase Material funciona como clase base de la jerarquía de recursos, mientras que Libro, Revista y Tesis son subclases que heredan atributos y métodos comunes.

Este uso de la herencia permite:

- reutilizar código compartido (título, autor, año, disponibilidad);
- extender el comportamiento según el tipo de material.

El polimorfismo permite que las operaciones sobre Material puedan aplicarse de manera uniforme a cualquiera de sus subtipos. Por ejemplo, un método que reciba un objeto Material puede trabajar indistintamente con un Libro, una Revista o una Tesis, facilitando la gestión de la biblioteca y reduciendo acoplamiento en el código.

Creación de materiales y uso de Factory Method

Para la creación de los distintos tipos de materiales (Libro, Revista, Tesis) se utiliza el patrón Factory Method. Este patrón permite encapsular la lógica de creación de objetos y devolver una instancia del tipo adecuado según un parámetro de entrada:

- 1 para Libro
- 2 para Revista
- 3 para Tesis

Gracias a esto, los controladores y la fachada pueden trabajar con la clase base Material sin conocer el subtipo concreto, promoviendo polimorfismo y reduciendo acoplamiento.

2.2.1. Comportamientos y restricciones del dominio

Además de definir la estructura básica de cada entidad, el modelo del dominio incorpora ciertos comportamientos y reglas relevantes para el funcionamiento del sistema:

Material

- Los materiales poseen un atributo de disponibilidad, el cual indica si pueden ser prestados o no.
- Su valor cambia automáticamente según el estado del préstamo asociado: Cuando un material es prestado, su disponibilidad pasa a “No disponible” y cuando el material es devuelto, vuelve a estar “Disponible”.

Esta regla garantiza que un material solo pueda estar prestado a un usuario a la vez, evitando conflictos o duplicidad en el registro de préstamos.

Préstamo

Un préstamo registra el estado de entrega del material y gestiona la información de devolución.

- Al crearse un préstamo, su estado de devolución se asigna como “No devuelto”, la fecha de devolución queda inicialmente vacía.
- Cuando el usuario devuelve el material, el sistema: Actualiza el estado a “Devuelto”, registra la fecha de devolución usando la fecha actual y notifica al material correspondiente para que recupere su disponibilidad.

De esta forma, la entidad Préstamo controla el ciclo de vida completo de una transacción, desde la entrega hasta el retorno del material.

2.2.2 Mapeo entre el modelo del dominio y la base de datos.

El modelo del dominio y la estructura física de la base de datos mantienen una relación cercana, pero no son idénticos. En particular, algunas entidades del dominio incluyen información adicional destinada a facilitar la visualización o el procesamiento dentro de la aplicación, mientras que la base de datos almacena únicamente los datos estrictamente necesarios.

En el caso de la entidad Préstamo, el modelo de dominio incorpora atributos como el nombre del usuario y el nombre del material, los cuales no existen como campos en la tabla correspondiente de la base de datos. En la estructura física, la información se representa únicamente mediante claves foráneas (`id_usuario`, `id_material`).

Para completar esta información, los DAOs aplican un proceso de mapeo durante las lecturas, utilizando consultas JOIN sobre las tablas relacionadas. De esta forma:

- PrestamoDAO recupera los registros de la tabla Prestamo.
- Mediante un JOIN con Usuario obtiene el nombre del usuario.
- Mediante un JOIN con Material obtiene el nombre del material.
- Con estos datos construye un objeto Prestamo del dominio que combina los campos almacenados en la base de datos, y los datos descriptivos necesarios para la capa de presentación.

Este proceso es útil para poder mostrar de manera más clara la información de un préstamo en la interfaz.

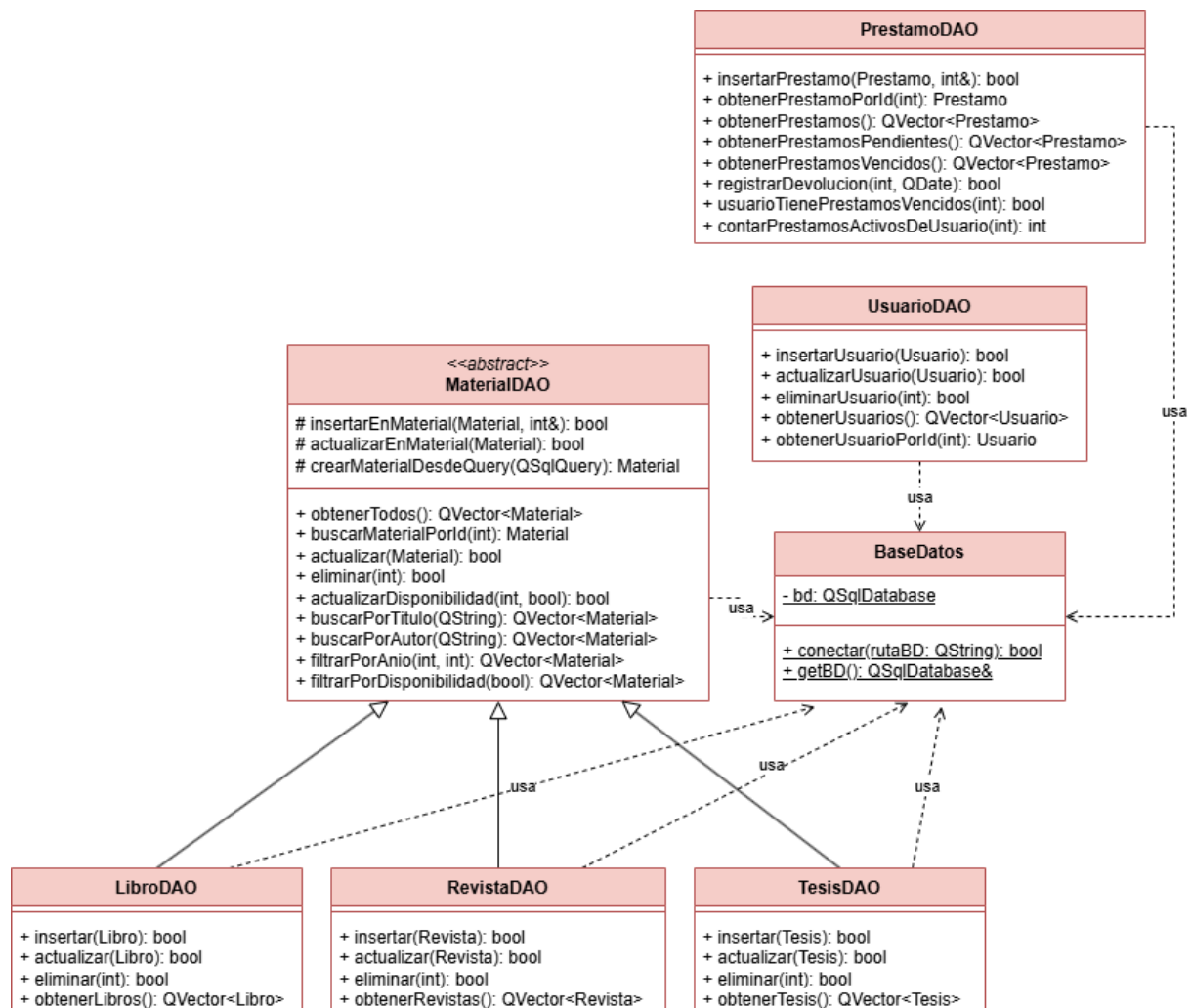
2.3. Capa de persistencia

La capa de persistencia es la responsable de gestionar el acceso a la base de datos y de garantizar que la información del sistema se almacene y recupere de manera segura y consistente. Su función principal es aislar al resto del sistema de los detalles concretos de almacenamiento, permitiendo que la lógica de negocio opere sobre objetos sin conocer cómo son guardados o consultados. Esta capa está compuesta principalmente por los Data

Access Objects (DAO) y por una clase auxiliar encargada de la conexión con la base de datos.

Responsabilidades principales

- Establecer y mantener la conexión con la base de datos.
- Ejecutar operaciones CRUD (crear, leer, actualizar y eliminar).
- Transformar filas de las tablas en objetos de dominio y viceversa.
- Manipular únicamente datos persistentes sin aplicar reglas de negocio.
- Servir como puente entre la lógica de negocio y la base de datos.



Componentes de la capa de persistencia

1. Clase BaseDatos

La clase **BaseDatos** centraliza la administración de la conexión al motor SQLite utilizado por el sistema.

Sus responsabilidades incluyen:

- Abrir la conexión mediante la ruta del archivo .db.

- Garantizar que exista una única instancia de conexión a través del método estático getBD().
- Proveer al resto de los DAOs acceso seguro y controlado a la base de datos.
- Esta clase no implementa operaciones CRUD; su propósito es únicamente preparar y exponer la conexión.

2. Data Access Objects (DAOs)

Cada entidad principal del sistema cuenta con un DAO responsable de interactuar con su tabla correspondiente:

- MaterialDAO
- UsuarioDAO
- LibroDAO
- RevistaDAO
- TesisDAO
- PrestamoDAO

Estas clases implementan las operaciones CRUD necesarias para la persistencia de datos, tales como:

- Registrar nuevos usuarios, materiales o préstamos.
- Consultar registros por ID o mediante filtros básicos.
- Actualizar información existente (por ejemplo, estado de devolución o disponibilidad).
- Eliminar registros de la base de datos.

Los DAOs trabajan únicamente con los modelos del dominio, recibiendo y devolviendo objetos simples (Usuario, Material, Prestamo, etc.).

No conocen ni interactúan con la lógica del sistema, los controladores o la fachada.

Relación con otras capas

- La capa de persistencia sólo se comunica con los controladores, quienes son los encargados de solicitar operaciones sobre la base de datos.
- No mantiene relación directa con la capa de presentación ni con la fachada.
- No accede al repositorio en memoria, ya que este pertenece exclusivamente a la lógica de negocio.

Este diseño asegura un sistema desacoplado, modular y fácil de mantener, en el que cada capa cumple un rol definido dentro del funcionamiento general del software.

2.3.1. Manejo de operaciones CRUD

Materiales y sus subtipos (Libro, Revista y Tesis)

Las operaciones CRUD para los distintos tipos de Material se gestionan mediante una combinación del MaterialDAO y el DAO específico del subtipo (LibroDAO, RevistaDAO, TesisDAO).

Agregar un material especializado:

- Se crea primero el registro base en la tabla de Material usando MaterialDAO.
- Una vez generado el ID del material, se utiliza para crear el registro específico en la tabla correspondiente al subtipo.

Actualizar un material especializado:

- Se modifica la información del material base (MaterialDAO).
- Luego se actualiza la información del subtipo (por ejemplo, LibroDAO).

Listar materiales:

- MaterialDAO consulta los registros de materiales.
- Según el valor del tipo almacenado, determina qué subtipo corresponde.
- Construye el objeto adecuado (Libro, Revista, Tesis).
- Devuelve una lista de objetos ya tipados correctamente.

Borrado lógico

La eliminación de materiales y usuarios se realiza mediante borrado lógico:

- No se elimina físicamente la fila.
- Se actualiza un campo de estado llamado "activo" para marcarlos como "inactivos" o "no disponibles" (0).

Esto permite conservar información histórica y mantener la integridad referencial con los préstamos existentes.

2.4. Capa de Lógica de Negocio

La capa de lógica de negocio es la encargada de coordinar las operaciones centrales del sistema, aplicar las reglas que gobiernan el funcionamiento de la biblioteca y orquestar la interacción entre la capa de presentación y la capa de persistencia.

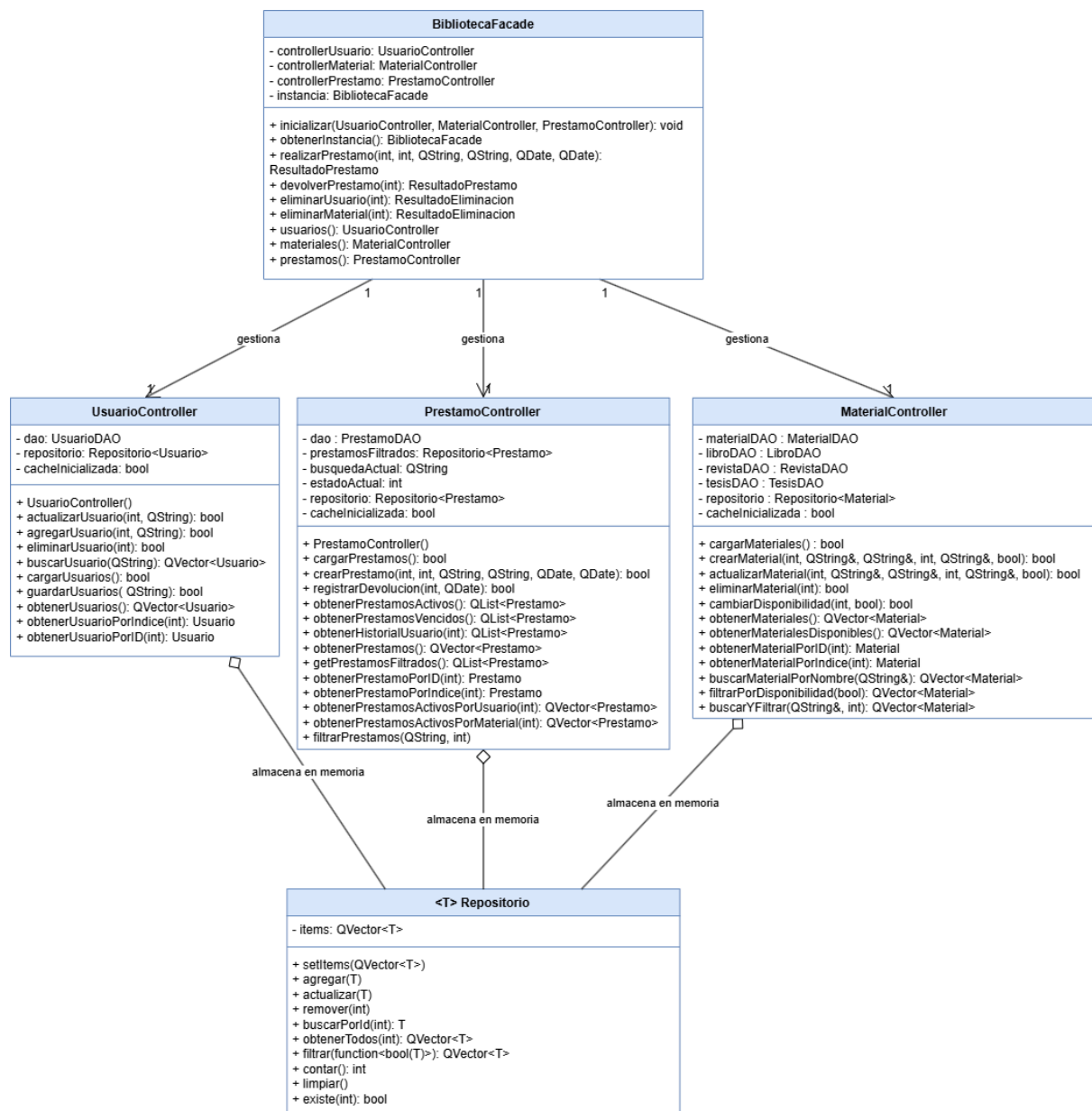
Su función principal es procesar las solicitudes provenientes de la interfaz de usuario, validar la información, aplicar restricciones del dominio y delegar las operaciones de almacenamiento a la capa de persistencia.

Esta capa está compuesta por los Controllers y por la clase Fachada (Facade), que actúa como un punto único de acceso para la mayoría de funcionalidades de alto nivel.

Responsabilidades principales

La lógica de negocio se encarga de:

- Coordinar el flujo de operaciones entre la base de datos y la interfaz de usuario.
- Aplicar las reglas del dominio, como validaciones de préstamo, disponibilidad de materiales o consistencia de estados.
- Interactuar con los DAOs a través de los controladores para ejecutar operaciones CRUD.
- Administrar el repositorio en memoria, que optimiza la recuperación de datos y reduce accesos directos a la base de datos.
- Evitar que la capa de presentación conozca detalles internos de persistencia.



2.4.1. Controladores (Controllers)

Cada entidad principal del sistema cuenta con un controlador específico, encargado de gestionar las operaciones relacionadas con dicha entidad:

- UsuarioController
- MaterialController
- PrestamoController

Los controladores cumplen las siguientes funciones:

- Validar datos de entrada antes de ejecutar una operación (ej., verificar disponibilidad de un material).

- Invocar los métodos correspondientes en los DAOs para realizar operaciones CRUD.
- Mantener sincronizado el repositorio en memoria con los datos de la base de datos.
- Transformar solicitudes de la interfaz en acciones concretas sobre el dominio.
- Garantizar que las reglas del sistema se cumplan antes de modificar el estado de las entidades.

Los controladores no acceden directamente a la interfaz de usuario ni al almacenamiento físico; su único enlace es con la fachada y los DAOs.

2.4.2. Fachada (BibliotecaFacade)

La clase Fachada actúa como un intermediario entre la interfaz gráfica y los controladores. Su objetivo es simplificar el uso del sistema proporcionando un único punto de acceso a las funciones principales.

Entre sus responsabilidades se encuentran:

- Unificar las operaciones de alto nivel (por ejemplo, registrar un préstamo completo).
- Reducir el acoplamiento entre la vista y los controladores..
- Exponer sólo los métodos necesarios para la interfaz, ocultando la complejidad interna.
- Exponer los métodos específicos de los controladores cuando sea necesario.
- Coordinar múltiples controladores cuando una operación involucra a más de una.

Por ejemplo, la creación de un préstamo involucra:

- Verificar disponibilidad del material.
- Registrar el préstamo en la base de datos.
- Actualizar el estado del material.
- Actualizar repositorios y notificar a la interfaz.
- Toda esta secuencia es gestionada desde la fachada.

2.4.3. Repositorio en memoria

Como mecanismo de optimización, la capa de lógica de negocio mantiene un repositorio interno en memoria que almacena temporalmente las listas de usuarios, materiales y préstamos cargados desde la base de datos.

Sus funciones incluyen:

- Reducir consultas repetitivas hacia la capa de persistencia.
- Agilizar filtrados, búsquedas y validaciones.
- Mantener consistencia local con los datos persistidos.
- Los controladores se encargan de mantener este repositorio sincronizado luego de cada operación CRUD.

2.4.4. Reglas de negocio aplicadas

La lógica de negocio implementa reglas que definen el comportamiento del sistema, entre ellas:

Restricciones a las operaciones sobre los registros:

- No se permite eliminar un material que tiene préstamos activos.
- Los usuarios con préstamos activos tampoco pueden ser eliminados.

Gestión de disponibilidad de materiales:

- Un material solo puede ser prestado si se encuentra marcado como “Disponible”.
- Al registrar un préstamo, el material pasa automáticamente a “No disponible”.
- Al registrar la devolución, el material recupera su disponibilidad.

Restricciones del ciclo de vida del préstamo:

- Un préstamo se crea con estado “No devuelto”.
- Solo se muestran materiales marcados como “disponible” y “activo”.
- Solo se muestran usuarios marcados como “activo”
- La fecha de devolución se asigna sólo al registrar el retorno.
- Tanto la fecha de préstamo como la fecha de devolución se asignan a la fecha actual y no se pueden modificar.
- No se permite devolver un material que ya fue marcado como devuelto.
- No se permite modificar ni eliminar los préstamos.

Integridad de entidades:

- No se permite eliminar usuarios o materiales físicamente para no quebrar la referencia con los préstamos.
- La eliminación se maneja mediante un borrado lógico administrado desde los controladores.

Reglas de validación de datos

- Los datos obligatorios deben estar completos para crear usuarios y materiales.
- Los identificadores recibidos deben ser válidos antes de realizar operaciones CRUD.

2.4.5. Relación con otras capas

La capa de lógica de negocio recibe solicitudes de la interface (por medio de la fachada).

- Coordina operaciones entre los diferentes controladores.
- Llama a los DAOs para persistir o consultar información.
- Mantiene la integridad del repositorio en memoria.

Este diseño asegura un flujo ordenado, evitando duplicación de lógica y promoviendo un sistema modular y fácilmente extensible.

2.5. Capa de Presentación

La capa de presentación corresponde a la interfaz gráfica desarrollada con Qt Widgets. Su función principal es permitir la interacción del usuario final con el sistema mediante ventanas, formularios y controles visuales.

Responsabilidades

- Mostrar información proveniente de la lógica de negocio.
- Recibir acciones del usuario (botones, formularios, búsquedas).
- Validar datos básicos antes de enviarlos al controlador (ej.: que los campos obligatorios no estén vacíos).
- Delegar toda la lógica de negocio a los controladores y a la fachada, sin procesar reglas internas.

Relación con otras capas

- No interactúa directamente con la base de datos.
- Solo se comunica con los controladores para ejecutar operaciones como registrar préstamos, listar materiales o gestionar usuarios.
- Recibe objetos ya procesados desde los controladores y los muestra en tablas, combos y formularios.

2.6. Patrones de Diseño y Técnicas de Programación Orientada a Objetos Utilizadas

En el desarrollo del sistema se aplicaron diversos patrones de diseño y técnicas de programación orientada a objetos con el objetivo de lograr una arquitectura modular, mantenible, escalable y fácil de comprender. A continuación, se describen los principales patrones y prácticas implementadas:

2.6.1. Patrón Factory Method

El patrón Factory Method se utilizó para centralizar la creación de objetos de la jerarquía Material, evitando que la vista o los controladores deban conocer los detalles concretos de cada tipo (Libro, Revista o Tesis).

Esta funcionalidad está implementada en la clase MaterialFactory, la cual es utilizada por el controlador de materiales para crear instancias especializadas de manera uniforme.

Este patrón permite:

- Ocultar la lógica de instanciación.
- Evitar el uso de condicionales repetidos.
- Simplificar la extensión del sistema si se agregan nuevos tipos de material.

2.6.2. Patrón Singleton

El patrón Singleton se aplicó en componentes que deben existir como instancia única durante toda la ejecución del programa:

- BaseDatos: asegura una única conexión global a SQLite.
- BibliotecaFacade: centraliza la lógica general del sistema a través de una única fachada.

El uso del Singleton evita la creación de instancias múltiples que podrían generar conflictos, y garantiza un acceso controlado y coherente a los recursos compartidos.

2.6.3. Patrón Facade

Para simplificar la interacción entre la interfaz gráfica (GUI) y la lógica interna, se utilizó el patrón Facade, implementado en la clase BibliotecaFacade.

Este componente:

- Expone operaciones simples a la vista.
- Oculta la complejidad de coordinar múltiples controladores.
- Reduce el acoplamiento entre la capa de presentación y la lógica del sistema.

Gracias a este patrón, las vistas trabajan con métodos directos, mientras que la lógica detallada permanece encapsulada en la fachada.

2.6.4. Herencia y Polimorfismo

El proyecto emplea herencia y polimorfismo para estructurar de manera coherente los diferentes tipos de materiales y sus operaciones de persistencia:

- Material actúa como clase base, de la cual heredan Libro, Revista y Tesis, permitiendo representar atributos y comportamientos comunes.
- MaterialDAO funciona como un DAO base que encapsula las operaciones generales de persistencia.
- Los DAOs especializados (LibroDAO, RevistaDAO, TesisDAO) heredan de MaterialDAO y añaden la lógica particular de cada subtipo.
- MaterialController coordina la lógica de negocio y, dependiendo del subtipo del material, invoca directamente al DAO específico correspondiente.
- Los DAOs especializados delegan las operaciones comunes en MaterialDAO, simulando así una operación en cascada.

Este uso de herencia y polimorfismo permite centralizar la lógica compartida, evitar duplicación de código y extender el sistema fácilmente añadiendo nuevos tipos de material sin afectar a la estructura ya existente.

2.6.5. Programación Genérica (Templates)

Se implementó un repositorio genérico basado en templates para almacenar y administrar entidades en memoria (usuarios, materiales, préstamos, etc.).

Este repositorio ofrece:

- Búsqueda por ID.
- Filtrado mediante lambdas.
- Inserción, actualización y borrado.
- Iteración sobre elementos.

El uso de programación genérica elimina duplicación de código y mejora la mantenibilidad siguiendo el principio DRY (Don't Repeat Yourself).

2.6.6. Encapsulamiento

Todas las clases del dominio mantienen sus atributos como privados, exponiendo únicamente getters y setters cuando es necesario.

El encapsulamiento garantiza:

- Protección del estado interno de los objetos.
- Validaciones antes de modificar datos sensibles.
- Separación clara entre estado interno y comportamiento externo.

Este principio evita inconsistencias y asegura coherencia con las reglas de negocio.

2.6.7. Uso de punteros inteligentes (std::shared_ptr)

Para una gestión segura de memoria dinámica se utilizan punteros inteligentes, especialmente std::shared_ptr, en controladores, repositorios, DAOs y modelos.

Sus principales beneficios son:

- Liberación automática de memoria, evitando fugas.
- Propiedad compartida, permitiendo que múltiples capas apunten al mismo objeto sin copias innecesarias.
- Compatibilidad con la arquitectura, ya que las entidades deben compartirse entre DAOs, repositorios y vistas.

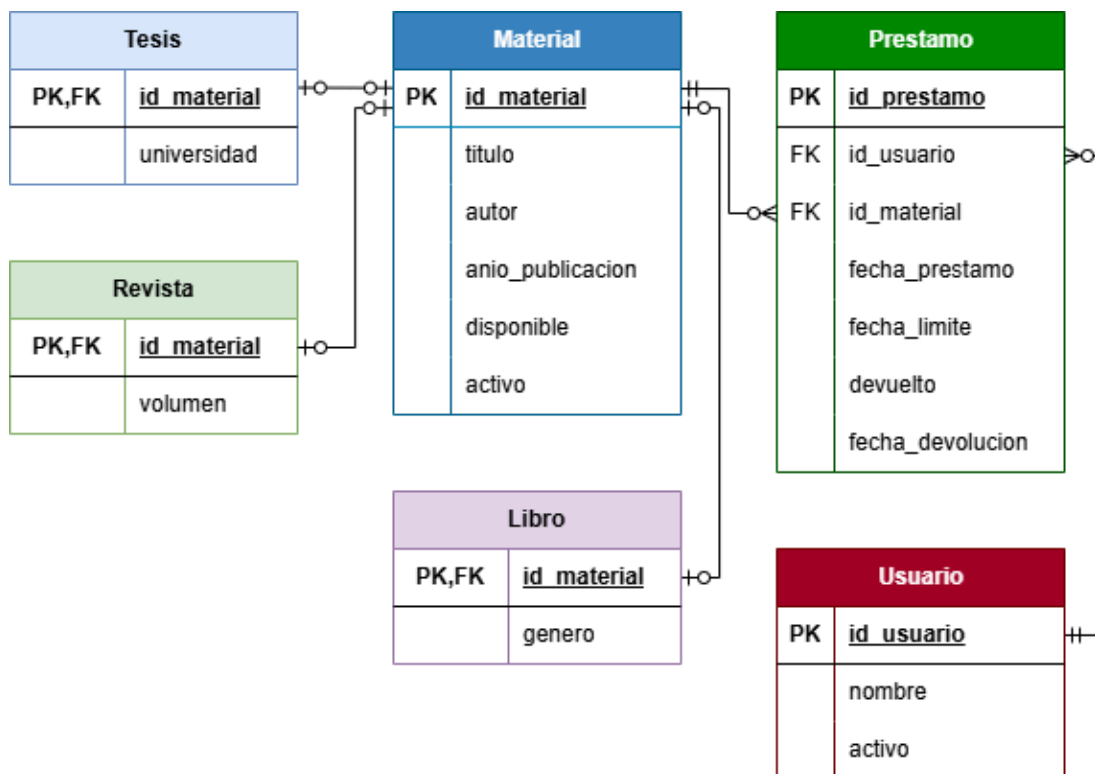
El uso de shared_ptr incrementa la seguridad, estabilidad y robustez del sistema, evitando errores comunes asociados a punteros crudos.

3. Diseño de Base de Datos

Para la persistencia de la información, el sistema utiliza una base de datos relacional SQLite. El diseño del esquema sigue las reglas de normalización para garantizar la integridad de los datos y minimizar la redundancia.

3.1 Modelo Entidad-Relación

El siguiente diagrama ilustra las entidades principales del sistema, sus atributos y las relaciones existentes entre ellas.



3.2 Estrategia de Generalización

Se ha implementado un patrón de Generalización/Especialización para el manejo del inventario bibliográfico.

- Entidad Padre (Material): Centraliza los atributos comunes a todas las publicaciones, como el titulo, autor, anio_publicacion y el estado de disponibilidad (disponible, activo).
- Entidades Hijas (Libro, Revista, Tesis): Contienen únicamente los atributos específicos de cada tipo de documento.

Esta relación se mantiene mediante la clave primaria id_material, que actúa simultáneamente como clave foránea en las tablas hijas, vinculando un registro específico (ej: una Tesis) con su información general en la tabla Material.

3.3 Diccionario de Datos

A continuación, se describen las tablas principales identificadas en el esquema:

A. Tabla Material: Representa el inventario físico de la biblioteca.

- disponible: Booleano que indica si el libro está físicamente en la biblioteca o prestado actualmente.
- activo: Permite el "borrado lógico" para no perder el historial de préstamos de libros dados de baja.

B. Tablas de Especialización

- **Libro:** Almacena el género literario o técnico de la obra.
- **Revista:** Almacena el número de volumen de la publicación.
- **Tesis:** Registra la universidad de origen del documento.

C. Tabla Usuario: Almacena la información de los usuarios de la biblioteca, tiene los campos nombre y activo (para el borrado lógico).

D. Tabla Prestamo (Transaccional): Es la tabla que registra los préstamos realizados mediante la relación entre un usuario y un material.

- id_usuario: ID del material prestado
- id_material: ID del usuario al que se prestó el material.
- fecha_prestamo: Fecha de registro del préstamo.
- fecha_limite: Fecha máxima de devolución.
- fecha_devolucion: Fecha en la que el usuario entregó el libro.
- devuelto: Indica si el material fue devuelto o no.

4. Funcionalidades del sistema

SisBib permite la gestión de préstamos y devoluciones así como los usuarios de la biblioteca y los materiales bibliográficos. Sus principales funcionalidades incluyen:

- Registrar nuevos materiales bibliográficos (libros, revistas y tesis).
- Editar información existente (título, autor, año, tipo, etc.).
- Eliminar materiales del registro.
- Listar todos los materiales disponibles.
- Guardar y cargar los datos desde una base de datos SQLite.
- Registrar usuarios que solicitan préstamos.
- Consultar y gestionar información de cada usuario.
- Realizar préstamos de materiales disponibles a usuarios registrados.
- Registrar la devolución de materiales prestados.
- Controlar el estado de cada material (disponible / prestado).
- Buscar materiales por título.
- Filtrar materiales según disponibilidad.
- Buscar usuarios por nombre.
- Buscar préstamos por nombre de usuario
- Filtrar préstamos según estado (pendientes, vencidos, devueltos).

5. Proceso de ejecución del proyecto

Clonar el repositorio

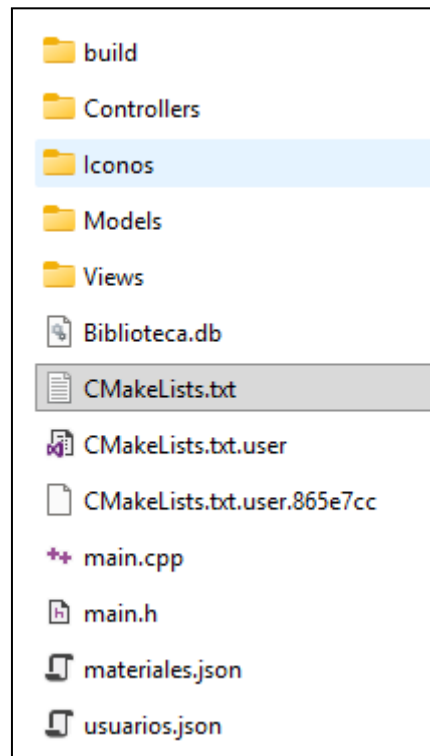
El código fuente puede obtenerse utilizando Git mediante el siguiente comando:

```
Shell  
https://github.com/rescobedoq/sbd.git
```

También puede descargarse en formato .zip desde la interfaz web de GitHub.

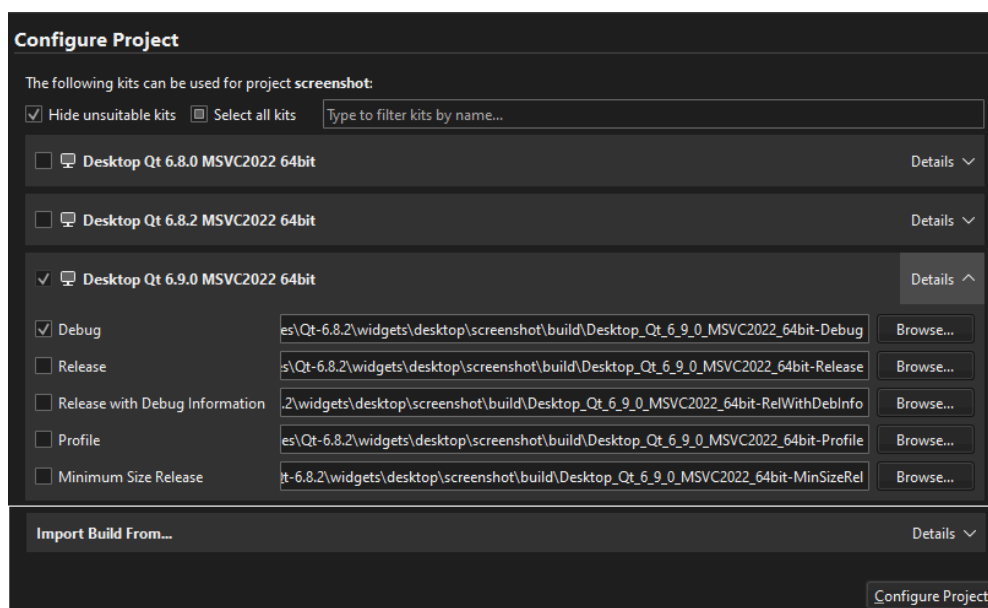
Abrir el proyecto en Qt Creator

Una vez descargado, debe abrir el archivo de proyecto dirigiéndose hacia File > Open File or Project en Qt y seleccionando el archivo CMakeLists.txt.



Qt Creator detectará automáticamente el tipo de proyecto y solicitará su configuración inicial. En la ventana de configuración, se debe seleccionar un kit de desarrollo compatible, por ejemplo:

- Desktop Qt 6.x.x MinGW 64-bit
- Desktop Qt 6.x.x MSVC



Qt Creator generará las carpetas de build necesarias.

Compilar el proyecto

Con la configuración completa, basta con presionar el botón Build o la tecla Ctrl + B. Qt Creator compilará automáticamente todos los módulos de la aplicación.

6. Conclusiones

La aplicación de escritorio SisBib constituye una solución integral para la gestión de usuarios, materiales y préstamos. Su diseño basado en la arquitectura Modelo–Vista–Controlador (MVC), complementado con componentes especializados como DAOs, repositorios en memoria y una fachada de coordinación, permite una estructura organizada, escalable y de fácil mantenimiento.